Cloud and On-Premises Deployment of Uzbek Legal RAG via Targeted Retriever Fine-Tuning

Tatul Danielyan **Mariam Avetisyan** **Hrant Davtyan**
Metric AI Lab
{tatul, mariam, hrant}@metricailab.com

arXiv:2608.29284v1 [cs.CL] 29 Aug 2026

Abstract

Deploying large language models for legal question answering raises challenges that general-purpose leaderboards do not capture, particularly for low-resource languages and under hard operational constraints. We report on building and operating a retrieval-augmented (RAG) legal assistant for Uzbek that must run in two regimes: a managed cloud service that maximizes answer quality within a per-token cost ceiling, and an on-premises deployment for clients whose legal data may not leave their infrastructure, restricting us to open-weight models on limited local hardware under latency constraints. Because no evaluation existed for this setting, we build two domain benchmarks: a retrieval benchmark of 178 expert-annotated legal queries with gold provision spans, and an end-to-end benchmark of 504 expert-curated question–answer pairs scored by an LLM judge whose ratings we validate against human judgments and against an independent-family judge. Applying these benchmarks under each regime, we find the open-versus-proprietary gap is small and cheaply closed by fine-tuning. Therefore, we train UTE-1, which is a state-of-the-art text embedder among open models for Uzbek. We also demonstrate that closing the performance gap via fine-tuning is both impractical due to the intensive hardware demands of long-context legal Q&A and unnecessary, given that legal acts change frequently. We support this by reporting a negative result from a QLoRA experiment. We distill practical guidance for similar deployments, drawn from a system serving real users in production. We release our benchmarks, evaluation code and the fine-tuned embedder (UTE-1) at this https URL to support future work on low-resource legal NLP.

1 Introduction

Large language models (LLMs) are increasingly deployed in legal and governmental settings, where citizens and professionals ask natural-language questions and expect answers grounded in the applicable statutes. Such systems are typically built as retrieval-augmented generation (RAG) pipelines (Lewis et al., 2020; Gao et al., 2023): a retriever selects relevant legal text, and an LLM composes an answer conditioned on it. RAG is a natural fit for the legal domain because the law is large, frequently amended, and demands attribution to specific provisions.

Most published guidance on building such systems is implicitly written for high-resource languages and a single, unconstrained deployment target. Real deployments are neither. We describe a production legal assistant for **Uzbek**, a Turkic language spoken by tens of millions but under-served by NLP resources, grounded in the national legal corpus (`lex.uz`). The system is commercially deployed and serves real users; per the double-blind policy we omit the product and organization names.

Two facts make model selection for this system non-obvious. First, the same assistant ships in *two deployment regimes* with conflicting constraints. The *cloud* regime is a multi-tenant SaaS offering: any model is permissible, but a per-token cost ceiling rules out the most expensive frontier APIs, and the objective is to maximize quality within that budget. The *on-premises* regime serves enterprise and public-sector clients whose legal documents *cannot leave their infrastructure*; data sovereignty restricts us to open-weight models, which must then run on the client's limited local hardware under latency constraints. Second, general multilingual leaderboards poorly predict quality here: strong aggregate ability does not imply strong Uzbek *legal* retrieval or answer quality, and the public Uzbek leaderboards that exist measure generic language ability, not grounded legal QA. Principled model choice therefore requires task- and language-specific evaluation that did not previously exist.

We make three contributions.

(1) A deployment case study with a transfer-

able decision procedure. We cast cloud and on-premises model selection as two constrained optimization problems and show how one evaluation suite drives both, yielding two deployed model stacks.

(2) Evaluation infrastructure for Uzbek legal QA. We release a 178-query retrieval benchmark with gold provision spans and a 504-item end-to-end QA benchmark, plus a *human-validated* LLM-as-judge protocol cross-checked against an independent-family judge—the linchpin of any such study. Both benchmarks and evaluation code are released.

(3) The finding that, under deployment constraints, the retriever is where to invest. Existing legal-RAG work fine-tunes the generator (Ma et al., 2025; Octadion, 2024), and parallel work (Butler and Butler, 2026) finds the retriever dominant but does not fine-tune it. We fine-tune an open embedder to a state-of-the-art open result for Uzbek (cheaply closing roughly half the in-domain gap to proprietary embeddings), and report a *negative* result: fine-tuning the generator under the same constraints is both costly to pursue and, given that laws change and multilingual LLMs keep improving, unnecessary. The heuristic—*fine-tune the retriever, rent or swap the generator*—is the paper's central, reusable lesson. The text embedder is released publicly to further facilitate NLP applications in Uzbek language.

2 System and Deployment Setting

Pipeline. The assistant is a RAG question-answering pipeline. Offline, we segment the national legal corpus into provision-level chunks, prepend each with its hierarchical headings (code/chapter/article) so locally ambiguous text stays interpretable, embed them, and index them in a vector database (Weaviate). At query time we normalize (and where needed translate) the question, then retrieve with *hybrid* search—dense cosine similarity (Karpukhin et al., 2020) fused with lexical matching—since legal queries hinge on exact terms (article numbers, statute names) that pure semantic search misses. We prompt an LLM with the retrieved provisions and a legal-assistant instruction to generate an answer citing the provisions it used. The production path retrieves $k{=}50$ chunks, so the retriever must ensure the governing provision is present in a 50-passage window (engineering details in Appendix C).

Why two models, and why they matter unequally. A RAG pipeline has two learnable components, the *embedder* (retriever) and the *generator*. Their failure modes are asymmetric: if the embedder fails to surface the governing provision, no generator can recover—the answer is confidently wrong or unsupported—whereas a capable generator mostly needs to read, synthesize, and attribute text it is given. This asymmetry motivates treating the embedder as a first-class object of optimization, a theme we return to in Sections 5 and 6.

Constraints. The retrieved context is large: in production, the $k{=}50$ provisions for a query span roughly **55k–90k tokens**. Two domain factors drive this: the same statute is cited across many articles, so a small top-k returns cross-references rather than governing text and high k is needed to capture them all; and statutory tables too large to embed as one unit are indexed as several chunks but re-expanded in full when any is retrieved. This long-context regime shapes both cost and latency. In the cloud regime, per-query cost is dominated by the generator's input tokens, so the cost ceiling effectively excludes frontier "pro"-tier APIs and admits only efficient "flash"-tier or open models. In the on-premises regime, the sovereignty constraint removes all API models, and latency is driven both by prefill over the large context and by decoding long legal answers that may require multi-step reasoning over several provisions—so prompt processing *and* decode time shape the user-visible wait.

3 Benchmarks and Evaluation Methodology

Because no public benchmark targets Uzbek legal RAG, and because the central risk in this study is trusting an unreliable evaluator, we invested first in evaluation. We build two complementary benchmarks and validate the automated judge against humans.

3.1 Retrieval benchmark

We construct a retrieval benchmark of 178 human-written and human-annotated legal questions[1]. For each question, domain experts identified the exact provision text required to answer it and recorded those passages as gold targets. We perform in-batch evaluation and report *top-5 retrieval accuracy*: whether at least one gold provision appears

[1]All questions are in Uzbek with Latin script.

among the five highest-ranked chunks. To situate our embedders externally, we also evaluate on the Uzbek subset of MTEB (Muennighoff et al., 2023) (127 queries over 1,263 candidates, nDCG@5). The two sets are independent in content and scoring and together cover 305 queries, measuring in-domain legal and general Uzbek retrieval respectively.

3.2 End-to-end QA benchmark

We curate 504 legal question–answer pairs spanning 22 legal domains [2]. 384 out of 504 (76%) are composed by legal experts (both questions and answers), 60 are sampled at random from search-style production queries (answers are expert-annotated), 60 are sampled at random from upvoted interactions (answers are LLM-drafted then reviewed, reformulated, and approved by experts).

Each item is scored by running the *entire production pipeline*: the question is retrieved against the live vector index (k=50) and answered by the generator, and the generated answer is compared against the reference. We score four quality dimensions with an independent LLM judge: *completeness*, *contextual accuracy* (faithfulness to retrieved provisions), *hallucination-free*, and *legal comprehension*, and report an *overall* score. Because it evaluates the deployed pipeline rather than models in isolation, the benchmark directly measures the quantity we care about and lets us vary the embedder and generator independently (Section 5).

3.3 LLM judge

Our headline QA metric is produced by an LLM judge. Using judge from the same family as responder LLM creates a risk for self-preference bias: LLM evaluators tend to favor their own family (Panickssery et al., 2024; Zheng et al., 2023) potentially inflating the shared-family generator's score. We therefore pick a capable LLM as judge which is from a completely different family than evaluated LLMs (`Muse Spark 1.1`).

We compare the chosen independent judge against human ratings. On a set of 52 questions scored on a 1–10 scale by domain experts, our judge attains Spearman ρ=0.7032 and a mean absolute error of 1.29 points against the expert scores.

[2] 312 questions are in Uzbek with Latin script, 91 are in Cyrillic Uzbek, and 101 are in Russian

4 Cloud Deployment: Quality Within a Cost Ceiling

In the cloud regime any model is admissible in principle, so selection reduces to maximizing quality subject to the cost ceiling.

Embedder. Table 1 reports retrieval quality for fourteen embedders. The proprietary `gemini-embedding-001` is best on our in-domain benchmark (0.961 top-5) and on MTEB (0.906 nDCG@5), with comfortable margins. As cloud embedding costs are a small fraction of total cost, we deploy it for the cloud service.

Generator. We pre-filter LLM candidates by three criteria: (i) sufficient general Uzbek ability, using a public Uzbek leaderboard (UzLib; full table in Appendix A); (ii) sufficient general capability, using public aggregate signals (e.g., arena rankings and MMLU-Pro); and (iii) price below our per-token ceiling.[3] This excludes the most expensive frontier APIs (e.g., the Claude, Gemini-Pro, and GPT-Pro tiers) despite their strong general scores. Among the in-budget candidates we then rank by end-to-end quality on our QA benchmark (Table 2). `gemini-3-flash` is the strongest in-budget generator (0.850 overall); the more expensive reference models we evaluate for context (e.g., `claude-sonnet-4-6`, `gpt-5.4`) do not surpass it on this task. We therefore deploy `gemini-embedding-001` + `gemini-3-flash` for the cloud service.

Cost and quality together. The cost ceiling is not a quality sacrifice. Plotting per-model inference cost against QA quality (Appendix B, Figure 1), `gemini-3-flash` sits at the quality-maximizing vertex of the cost–quality Pareto frontier and *strictly dominates* every frontier API: it is 2.1×, 4.0×, and 6.6× cheaper than `grok-4.3`, `gpt-5.4`, and `claude-sonnet-4-6` respectively, while scoring higher than all three. For this task, "rent the flash tier, not the frontier" is the cost–quality-optimal choice, not a budget compromise.

Production corroboration. The deployed cloud stack serves ≈30k questions per month. Over 479

[3] Our ceiling is $3 per million input tokens and $15 per million output tokens—a business decision balancing answer quality against competitive end-user pricing and local purchasing power. Because retrieved legal context runs to 55–90k tokens per query (§2), per-query cost is dominated by input tokens, so the input-token price is the binding figure; the answer itself is a small fraction.

in-product votes, 78% were positive (375/479), corroborating the offline QA ranking that selected this stack. The on-premises deployments surface *no* such telemetry by design: the same data sovereignty that motivates them withholds the feedback signal the cloud service relies on, a tradeoff we revisit in §6.

5 On-Premises Deployment: Sovereignty, Open Weights, and Latency

The on-premises regime removes API models entirely. Selection becomes: among open-weight models that fit the client's hardware, maximize quality subject to a latency budget. We address the retriever and generator in turn, and then explain why we did *not* fine-tune the generator.

5.1 Closing the retriever gap by fine-tuning (UTE-1)

Table 1 shows that the best open embedder out of the box (`arctic-embed-l-v2.0`) trails the proprietary leader by a meaningful margin on our in-domain benchmark (0.876 vs. 0.961 top-5). Given the asymmetry argued in Section 2, we judged this gap worth closing, and embedders are cheap to adapt relative to LLMs. We fine-tuned `arctic-embed-l-v2.0` into **UTE-1** from two data sources: (i) ≈10k noisy synthetic translations of community QA data, following a recipe of Navasardyan et al. (2026) for adapting embedders to low-resource languages with deliberately imperfect supervision; and (ii) ≈7k question–chunk pairs mined from positively-rated production interactions of the cloud service, providing in-domain legal positives. An ablation isolates the two sources: training on the 10k synthetic pairs *alone* already yields a strong open embedder (in-domain top-5 0.90), and adding the 7k production-mined real pairs improves it further to 0.916 — our final UTE-1, which is state-of-the-art among open embedders on both our benchmark and MTEB[UZ] (Table 1), +0.040 top-5 over its base and closing roughly half the in-domain gap to the proprietary leader. This is the empirical core of our "invest in the retriever" thesis: a cheap fine-tune with synthetic data plus a little real in-domain signal recovers roughly half of the retrieval quality that separates open from proprietary.

5.2 Choosing the generator: quality, then latency

Among open generators (Table 2), `gemma-4-31b-it` is the strongest open model overall and `qwen3.6-35b-a3b` the second (0.724 vs. 0.671 with the UTE-1 retriever). We nonetheless deploy the Qwen model, because the on-premises objective is quality *subject to a latency budget*, and the two models sit at very different points on the compute frontier: `gemma-4-31b` is a dense model that activates all 31B parameters per token, whereas `qwen3.6-35b-a3b` is a mixture-of-experts model (Fedus et al., 2022) that activates only ≈3B parameters per token.

We profiled the top 2 open candidates on a single-tenant H200 hardware over the 504-query benchmark (Table 3). Both models are inferences using same vLLM ((Kwon et al., 2023)) setup paired with their official MTP speculative decoding drafter models. Results show, that the 3B-active Qwen model produces both the first token (TTFT p50) as well as the full response (Total p50) faster than dense Gemma while producing more reasoning tokens.

This matters when serving on-premises as the 3B-active model sustains higher throughput and more concurrent users at lower cost than the 31B-dense one - a large efficiency gain for a modest quality concession. We therefore deploy `qwen3.6-35b-a3b` on-premises.

5.3 Why we did *not* fine-tune the generator (a negative result)

A natural next step is to fine-tune the open generator on Uzbek legal QA. Under our resource constraints the only feasible method is QLoRA (Dettmers et al., 2023; Hu et al., 2022). We attempted to QLoRA-fine-tune an open MoE model (`gemma-4-26b-a4b`) on 8 AMD GPUs, and abandoned the effort for reasons we think are instructive.

It is impractical. Faithful RAG training requires conditioning on the retrieved context, but our contexts run to 55–90k tokens (§2). The obstacle is not a hard capacity wall—one can always trade batch size, sequence packing, or checkpointing for memory—but economics: every training and evaluation step processes these very long sequences, turning each experiment into a slow, expensive run and the hyperparameter search into a low-throughput loop we could not justify. The

	Embedder	Open	Retr. Top-5	MTEB nDCG@5	Avg
1	`gemini-embedding-001` (3072d)	—	**0.961**	**0.906**	**0.934**
2	`gemini-embedding-2` (3072d)	—	0.938	0.900	0.919
3	**UTE-1 (ours)**	✓	0.916	0.863	0.889
4	`Qwen3-Embedding-4B`	✓	0.893	0.795	0.844
5	`arctic-embed-l-v2.0`*	✓	0.876	0.850	0.863
6	`multilingual-e5-large`	✓	0.832	0.763	0.797
7	`multilingual-e5-large-inst.`	✓	0.815	0.811	0.813
8	`jina-embeddings-v5-small`	✓	0.803	0.777	0.790
9	`multilingual-e5-base`	✓	0.753	0.718	0.735
10	`Qwen3-Embedding-0.6B`	✓	0.685	0.743	0.714
11	`harrier-oss-v1-0.6b`	✓	0.635	0.758	0.696
12	`harrier-oss-v1-270m`	✓	0.612	0.756	0.684
13	`geevec-embeddings-1.0-lite`	✓	0.528	0.506	0.517
14	`embeddinggemma-300m`	✓	0.489	0.695	0.592

Table 1: Embedder retrieval quality on our in-domain legal benchmark (top-5 accuracy over 178 queries, §3.1) and the Uzbek MTEB subset (nDCG@5 over 127 queries / 1,263 candidates), sorted by the in-domain metric. **Bold** = best overall; underline = best open-weight. *Base model for UTE-1: fine-tuning lifts in-domain top-5 from 0.876 to 0.916 (+0.040) and MTEB from 0.850 to 0.863, closing roughly half of the remaining gap to the proprietary leader.

Generator	Open	Retriever	Overall	Compl.	Ctx. Acc.	Halluc.-Free	Legal Compr.
`gemini-3-flash` •	—	`gemini-001`	**0.850**	**0.831**	**0.889**	0.772	**0.907**
`claude-sonnet-4-6` °	—	`gemini-001`	0.839	0.758	0.875	0.869	0.853
`gemini-3.1-flash-lite` •	—	`gemini-001`	0.817	0.653	0.885	0.911	0.821
`gpt-5.4` °	—	`gemini-001`	0.809	0.659	0.815	**0.990**	0.772
`gemma-4-31b-it` *	✓	`gemini-001`	0.803	0.679	0.849	0.899	0.786
`gemma-4-31b-it` *	✓	`UTE-1`	0.724	0.534	0.788	0.879	0.694
`qwen3.6-35b-a3b` *	✓	`gemini-001`	0.721	0.661	0.774	0.687	0.762
`qwen3.6-35b-a3b` *	✓	**UTE-1**	0.671	0.563	0.728	0.710	0.681
`gemma-4-26b-a4b-it` *	✓	`gemini-001`	0.665	0.520	0.687	0.810	0.643
`grok-4.3` °	—	`gemini-001`	0.593	0.333	0.635	0.929	0.476
`gemma-4-26b-a4b-it` *	✓	`UTE-1`	0.570	0.397	0.589	0.784	0.510

Table 2: End-to-end QA quality (§3.2), scored on all 504 items by the third-family judge `meta/muse-spark-1.1`: no Meta model appears in the candidate set or deployed pipeline. Rows are ordered by Muse overall score. •in-budget cloud candidate; °frontier API shown as a reference point but excluded by the cost ceiling; *open-weight, on-premises candidate. The **deployed** stacks are `gemini-3-flash`+`gemini-001` (cloud) and `qwen3.6-35b-a3b`+UTE-1 (on-premises, see §5).

Model	TTFT p50	Total p50	Decode tok/s	Out tok p50
`qwen3.6-35b-a3b`	**2.4 s**	**29.7 s**	**171**	4944
`gemma-4-31b`	14.7 s	63.9 s	18	736

Table 3: Latency of the top two open candidates over the 504-query benchmark, measured on a single-tenant H200 hardware. MoE Qwen model is favored by all 3 latency measures while producing more reasoning tokens.

tempting shortcut—dropping the context to shorten inputs—trains the model to answer legal questions *without* retrieved provisions, exactly the behavior that causes hallucination in deployment, so it is not an acceptable fix.

It is also unnecessary. Even if we could train it, what would it teach? (i) Better Uzbek: plausibly out of reach for a light QLoRA adapter and, more importantly, a depreciating investment, since base multilingual LLMs improve release over release. (ii) The content of the law: this would require continual pretraining, not a small adapter—and laws are amended, so any weights-baked legal knowledge is stale on arrival. The correct abstraction for changing law is retrieval, not memorization. This is the second half of our thesis: the generator's gap is better addressed by *renting or swapping* a stronger model than by fine-tuning, whereas the retriever's gap is worth closing once and reusing. As multilingual base models keep improving release over release, the case for adapter-tuning a generator for this task only weakens further.

5.4 Reducing latency further

Since latency is the binding on-premises constraint, it is where we spend additional effort. Both candi-

date open families natively support *multi-token prediction* (MTP), a self-speculative scheme in which auxiliary heads propose future tokens the model verifies in parallel (Leviathan et al., 2023; Chen et al., 2023). We serve `qwen3.6-35b-a3b` with its MTP head drafting two tokens per step, a further $\approx 1.2\times$ decode speedup at no quality change—stacking with the active-parameter advantage.

6 Discussion: Lessons Learned

In low-resource RAG, the retriever is the higher-leverage investment. The open/proprietary gap was smaller for retrieval than generation and far cheaper to close—a modest embedder fine-tune (§5.1) recovered roughly half of it, while generator fine-tuning was costly and of dubious value (§5.3)—so on a limited budget, spend it on the embedder first.

Build domain evaluation before choosing models, and validate the judge. General leaderboards did not predict in-domain legal quality, and our entire selection rests on an automated judge. Spending first on a small, expert-grounded benchmark and a human-validated judge protocol (Section 3.3) was the precondition for every subsequent decision.

Digraphia is a concrete hazard general benchmarks miss. Uzbek is written in both Latin and Cyrillic, and our deployment surfaced *script switching* (e.g. a Cyrillic question drawing a Latin answer, or one answer mixing scripts) driven by Latin-skewed pretraining and a mixed-script corpus whose retrieved context can span both scripts (23 benchmark items).

The impact is more severe in citations. Script switching inside a law title or article heading can break or misdirect a citation. This affects 5.08% of law citations pre-normalization. Given that citation of the relevant legal articles is a critical component of the product, we post-process results with a deterministic rule-based Latin↔Cyrillic transliteration applied to Uzbek spans only (Russian text which also uses Cyrillic script is untouched). The post-processing is content-preserving and resolves all such observed cases fixing broken citations.

Sovereignty is a first-class constraint—and it costs observability. For legal and public-sector clients, "open weights, on-premises" is a requirement, not a preference, and it is what made the open-model and latency work necessary. It also withholds telemetry: the cloud service gives an online signal (§4) that agrees with the offline ranking, but on-premises deployments return none—the data that cannot leave cannot be measured—raising the stakes on getting offline evaluation right before shipping.

7 Conclusion

We described a production Uzbek legal RAG assistant deployed across cloud and on-premises regimes, where two benchmarks and a validated judge drove principled model selection under cost, sovereignty, and latency constraints. The transferable lesson: in low-resource legal RAG, invest in the retriever (fine-tune once, reuse) and rent or swap the generator rather than fine-tune it.

Limitations

Our study covers a single language (Uzbek), a single domain (statutory law), and a single jurisdiction; the specific model rankings will not transfer, though we believe the decision procedure and the retriever-first lesson are more general. The benchmarks are modest in size (178 retrieval queries; 504 QA items) and the judge-validation set is small (52 items); we therefore avoid over-interpreting small score differences and will release the data so others can extend it. Each item was annotated by a single domain expert, so we cannot report inter-annotator agreement; we mitigate this by using qualified legal professionals and by cross-validating the automated judge against both human scores and a family-independent judge (§3.3), but the lack of multiply-annotated items is a genuine limitation. Our headline quality metric is produced by an LLM judge; although we chose an independent judge and validated it against humans, potential bias (e.g., shared pretraining data or stylistic preferences not captured by our held-out set) cannot be ruled out, and the human study itself is limited in scale. The negative QLoRA result is specific to our hardware, our context lengths, and the methods we could afford; it shows that this path was not worth the cost of iteration *for us*, not that generator fine-tuning is infeasible in general. Model identifiers refer to versions current at the time of writing; given the pace of releases, absolute numbers will date quickly, and several reported latency, cost, and judge-correlation figures are deployment-specific and should be read as characterizing our setting rather than as universal constants.

Acknowledgements

The authors would like to express their gratitude to the BalcomSoft and Tuzuk.AI teams, with special thanks to Avazbek Nuriddinov and Askar Djumanov for their invaluable support. This work was made possible through their generous provision of data, domain expertise, and high-quality annotations.

Ethical Considerations

The system answers legal questions but is an information-retrieval aid, not a substitute for qualified legal counsel; deployed interfaces present answers with citations to source provisions and with disclaimers to that effect, and we caution against using outputs as definitive legal advice. Hallucination is an acute risk in this domain, which is precisely why we (i) ground answers in retrieved statute, (ii) measure a dedicated hallucination-free dimension, and (iii) chose RAG over weights-baked legal knowledge so that answers track the current law. The on-premises offering exists to honor data-sovereignty and confidentiality requirements of legal and public-sector clients, keeping sensitive documents on client infrastructure. Production interaction data used to build training sets (Section 5.1) was limited to positively-rated exchanges and handled under the product's terms of service; we use it in aggregate to mine question–provision relevance, not to expose individual users. Expert annotators contributing reference answers and human judgments were compensated domain professionals. Because the legal corpus and the assistant operate in a specific national context, deployment elsewhere would require re-grounding in the relevant law.

References

Abdur-Rahman Butler and Umar Butler. 2026. Legal rag bench: An end-to-end benchmark for legal rag. *arXiv preprint arXiv:2603.01710*.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. *Preprint*, arXiv:2309.06180.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 19274–19286.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Hanjie Ma, Yuhang Lu, Zhengdong Xiao, Jie Feng, Haixiang Zhang, and Jian Yu. 2025. Sdd-lawllm: Advancing intelligent legal systems through synthetic data-driven fine-tuning of large language models. *Electronics*, 14(4):742.

Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2023. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 2014–2037.

Zaruhi Navasardyan, Spartak Bughdaryan, Bagratuni Minasyan, and Hrant Davtyan. 2026. Less is more: Adapting text embeddings for low-resource languages with small scale noisy synthetic data. In *Proceedings of the Workshop on Low-Resource Language Models (LoResLM) at EACL 2026*.

One Octadion. 2024. Asvri-legal: Fine-tuning llms with retrieval augmented generation for enhanced legal regulation. *arXiv preprint arXiv:2511.03563*.

Arjun Panickssery, Samuel R. Bowman, and Shi Feng. 2024. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

A UzLib Leaderboard (Candidate Pre-Filtering)

We use the public UzLib leaderboard of general Uzbek language ability as the first of three pre-filters on LLM candidates (Section 4). UzLib measures generic language ability rather than grounded legal QA, so we treat it only as a coarse admissibility filter, not as a selection metric; final selection uses our end-to-end QA benchmark (Table 2). The full leaderboard, abbreviated to the entries relevant to our candidate set, is reproduced from the public source; we omit it here for space and will include it in the camera-ready appendix.

B Cost–Quality Analysis

This appendix expands the cost–quality result summarized in Section 4. For each generator we measured the cost of answering the full 504-query QA benchmark through each provider's standard (non-batch) API (Table 4); standard pricing is what production incurs. We use *inference* cost as the deployment-relevant axis. The cost of the LLM judge is a separate, roughly constant evaluation overhead (≈$22 per model, since the same judge scores every model's 504 answers) and is excluded from the deployment comparison.

Figure 1 plots inference cost (log scale) against overall QA score (`gemini-001` retriever, for a common comparison). The Pareto frontier runs through `gemma-4-26b-a4b`, `gemma-4-31b`, `gemini-3.1-flash-lite`, and `gemini-3-flash`. Two readings matter.

For the cloud regime, `gemini-3-flash` is the quality-maximizing vertex of the frontier and *strictly dominates* all three frontier APIs: each costs more *and* scores lower (`grok-4.3` 2.1×, `gpt-5.4` 4.0×, `claude-sonnet-4-6` 6.6× the cost). This is the empirical backbone of "rent the flash tier, not the frontier."

For the on-premises regime, this cost axis does *not* carry over. Per-token hosted pricing reflects cloud economics; on owned hardware the binding cost is serving throughput, governed by active parameters (§5.2)—which is why the deployed on-premises model (`qwen3.6-35b-a3b`, 3B active) differs from the hosted-API cost–quality pick among open models (`gemma-4-31b`). The figure is a cloud-regime tool; the same "hosted metrics mislead on-prem" caveat from latency (§5.2) applies to cost.

Generator	QA	Inf. cost ($)
`gemini-3-flash`•	**0.850**	21.72
`claude-sonnet-4-6`°	0.839	142.63
`gemini-3.1-flash-lite`•	0.817	11.24
`gpt-5.4`°	0.809	87.02
`gemma-4-31b-it`⋆	0.803	4.36
`qwen3.6-35b-a3b`⋆	0.721	8.48
`gemma-4-26b-a4b-it`⋆	0.665	2.37
`grok-4.3`°	0.593	46.05

Table 4: Overall QA score with the `gemini-001` retriever, scored on all 504 queries by the third-family judge `meta/muse-spark-1.1`, and generator-inference cost at each provider's standard list rate.

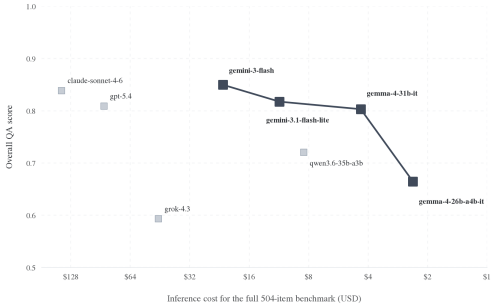


Figure 1: Cost–quality landscape (Appendix B). Top-right is better. `gemini-3-flash` achieves highest quality on the Pareto frontier. Among open models, the on-premises pick (`qwen3.6-35b-a3b`) is chosen on serving throughput, not this hosted-API cost axis (§5.2).

C Reproducibility Details

Production pipeline. Chunks are segmented at provision granularity and prepended with their hierarchical headings (code/chapter/article) to preserve context; oversized statutory tables are indexed as multiple chunks but re-expanded in full whenever any chunk is retrieved. Retrieval is hybrid (dense + lexical), preceded by question normalization and, where needed, translation; answers are post-processed to normalize script (Latin/Cyrillic) to match the question (§6). Training and evaluation sets are de-duplicated with both semantic and fuzzy lexical matching.